

Anders Hejlsberg



Warum?

- Turbo Pascal
- Delphi
- .NET / C#
- TypeScript



Verlauf

- Geburt
- Leben
- ~~Tod~~



Technische Daten

Name	Anders Hejlsberg
Geburtstag	2.12.1960 (64 Jahre)
Ort	Copenhagen, Denmark
Studium	Engineering @ Technische Universität Dänemark





1980 - 1982: Pascal

- Turbo Pascal
- Borland
- Erste IDE



1991: Delphi

- Windows
- Delphi
- Chefentwickler



```
program ArrayLoop;  
{$APPTYPE CONSOLE}  
const a : array[1..3] of real = ( 1.1, 2.2, 3.3 );  
var f : real;  
begin  
    for f in a do  
        WriteLn( f );  
    end.  
end.
```


1996: Angebot

- Microsoft: .5 Mio
- Borland: Kontenangebot
- Microsoft: verdoppelt
- Wechsel Oktober 1996



1996+: Visual J++

- Microsofts Java Implementierung
- Unzureichende JVM Kompatibilität
- Klage von Sun





.NET

2000: .NET Framework

- C++, VisualBasic, J++
inkompatibel
- Gemeinsame basis



2000: C#

- Leitender Designer
- Einfachheit von VisualBasic
- Stärke von C++



2001

Dr. Dobb's Excellence in Programming
Award

*"Über arbeiten an TurboPascal bis C#,
hat Anders Hejlsberg signifikante
Beiträge zu der Kunst und der
Wissenschaft des Programmierens
getätigt." [1]*





C#

- 2005: Generics, partielle Typen, ...
- 2008: Extension members, LINQ, ...
- 2012: async, await, ...

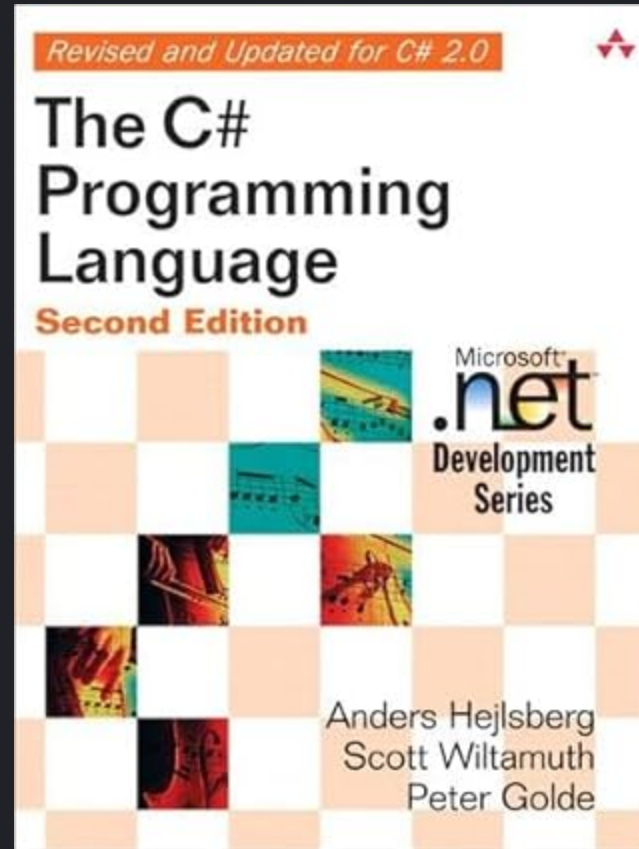


```
public static class MyClass<T>
{
    public static Type MyType() { return typeof(T); }

    // LINQ / extension member Beispiel
    public static IEnumerable<T> MyFunc(this IEnumerable<T> collection) {
        return collection
            .Where(x => ...)
            .Select(x => ...)
            .OrderBy(x => ...);
    }

    // async / await Beispiel
    public static async Task<string> MyAsyncFunc(HttpClient client) {
        var result = await client.GetAsync("example.com");
        if (!result.IsSuccessStatusCode) { ... }
        return await result.Content.ReadAsStringAsync();
    }
}
```

Buch: "The C# Programming Language"



ab 2010

- Umstellung auf Roslyn Compiler
- Open Source
- Langeweile
- #Script





TypeScript

- JavaScript mit Typen
Compiler
- C
- Objektorientiert
- Funktional /
Javascript style



```
type ID = string;
interface Task { id: ID; title: string; done: boolean }

const makeId = () => Math.random().toString(36).slice(2,8);

class Store {
  private tasks = new Map<ID,Task>();
  add(title: string){
    const t: Task = { id: makeId(), title, done: false };
    this.tasks.set(t.id,t);
    return t;
  }
  list(){ return [...this.tasks.values()]; }
  complete(id: ID){ const t = this.tasks.get(id); if(t) t.done = true; return t; }
}

(async()=>{
  const s = new Store();
  const a = s.add("Write slide");
  s.add("Review");
  s.complete(a.id);
  console.log(s.list());
})();
```


Person

- Neutral
- Zuhörer

